

Base de données — Documentation

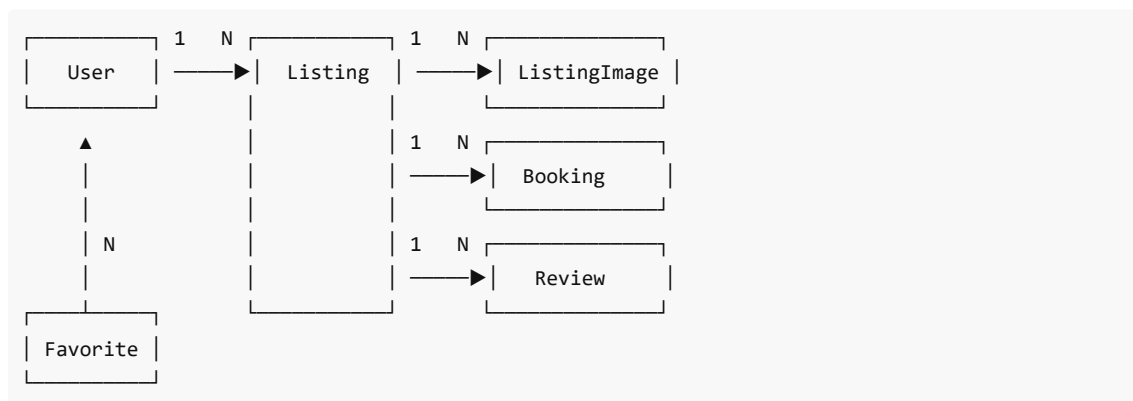
Choix techniques

La base de données d'ETNAir est une instance **PostgreSQL 16**, choisie pour sa robustesse, ses fonctionnalités avancées (recherche full-text, JSON, géolocalisation native via PostGIS si besoin) et son large écosystème. L'accès au SGBD passe exclusivement par **Prisma 6**, un ORM moderne qui combine excellente expérience développeur et performances solides.

Le projet n'écrit volontairement aucune requête SQL brute : toute la couche d'accès aux données passe par les méthodes typées de Prisma (`prisma.user.findUnique` , `prisma.listing.findMany` , etc.). Cela garantit que le schéma reste cohérent entre le code et la base, et que les migrations sont gérées de manière déclarative via le fichier `schema.prisma` .

Modèle de données

Le modèle relationnel articule six entités principales qui correspondent aux concepts métier de la plateforme. Un **utilisateur** peut être soit locataire (`tenant`), propriétaire (`owner`), ou administrateur (`admin`). Un propriétaire peut publier plusieurs **annonces** (listings), chacune comportant des **images** (listingImages), pouvant faire l'objet de **réservations** (bookings), recevant des **avis** (reviews), et étant éventuellement mise en **favori** par d'autres utilisateurs.



Le modèle `User`

L'entité `User` représente n'importe quel utilisateur de la plateforme. Elle contient les informations classiques : un identifiant auto-incrémenté, un prénom, un nom, une adresse email unique, un mot de passe hashé via bcrypt et un rôle qui prend l'une des trois valeurs `tenant` , `owner` ou `admin` . Une date de création est automatiquement renseignée. Côté relations, un utilisateur peut avoir des annonces (s'il est propriétaire), des réservations (s'il est locataire), des favoris et des avis.

Le modèle `Listing`

L'entité `Listing` représente une annonce de logement. Elle contient un titre, une description, une ville, un prix par nuit (en `Decimal` avec deux chiffres après la virgule pour éviter les erreurs de précision flottante), des coordonnées GPS optionnelles (latitude et longitude), et des dates de disponibilité. Chaque annonce est rattachée à un propriétaire via la clé étrangère `ownerId` , avec une suppression en cascade : si le propriétaire supprime son compte, ses annonces sont supprimées.

Les relations inverses permettent d'accéder facilement aux images, réservations, avis et favoris de l'annonce.

L'inclusion d'un `_count` avec un `where` permet de calculer dynamiquement le flag `isBooked` lors de la

récupération de la liste d'annonces.

Le modèle `ListingImage`

Cette table stocke les images uploadées par les propriétaires. Chaque ligne contient un `url` (en réalité un chemin relatif vers `/uploads/<filename>`), une `key` qui correspond au nom du fichier sur disque, et un `listingId` qui fait le lien avec l'annonce. La suppression cascade garantit que les références en base sont nettoyées quand une annonce est supprimée. Les fichiers eux-mêmes restent sur le disque et nécessitent un job de nettoyage périodique (non implémenté actuellement).

Le modèle `Booking`

L'entité `Booking` enregistre une demande de réservation. Elle stocke l'utilisateur locataire, l'annonce concernée, les dates de début et de fin, le prix total calculé (`nights × pricePerNight`), et un statut qui prend l'une des valeurs `pending`, `confirmed` ou `cancelled`. Un champ `cancelDeadline` est calculé automatiquement à la création (48 heures avant la date d'arrivée) et sert de référence pour autoriser ou refuser une annulation tardive.

Le modèle `Favorite`

`Favorite` est une simple table de jointure entre `User` et `Listing`, avec une contrainte d'unicité `@@unique([userId, listingId])` qui empêche un utilisateur de mettre la même annonce en favori plusieurs fois. La suppression cascade dans les deux sens (utilisateur ou annonce) garantit la cohérence référentielle.

Le modèle `Review`

L'entité `Review` représente un avis laissé sur une annonce. Elle contient une note de 1 à 5, un commentaire textuel optionnel, et les références à l'utilisateur auteur et à l'annonce concernée. À noter : sur le frontend actuel, les avis sont stockés en `localStorage` pour la démo, le modèle existe en base mais n'est pas encore branché à des endpoints REST. C'est un point d'évolution prévu.

Migrations

Toutes les migrations sont stockées dans `api/prisma/migrations/` sous forme de fichiers SQL générés par Prisma. Chaque dossier porte un timestamp et un nom descriptif. La migration initiale `20260512083611_init` crée les tables `User`, `Listing`, `Booking` et `Review`. La migration `20260522091103_add_coordinates` ajoute les champs `latitude` et `longitude` au modèle `Listing` pour permettre l'affichage sur la carte. La migration `20260528000000_add_listing_images` crée la table `ListingImage` (auparavant les images étaient stockées sur MinIO et référencées différemment). Enfin, `20260528120000_add_favorites` crée la table `Favorite`.

Pour appliquer toutes les migrations en attente sur une base existante, il suffit d'exécuter `docker compose exec api npx prisma migrate deploy`. Cette commande est idempotente et ne réapplique pas les migrations déjà passées. Elle est aussi appelée automatiquement dans le `docker-entrypoint.sh` à chaque démarrage du conteneur, ce qui garantit que le schéma de la base est toujours synchronisé avec le code.

Pour créer une nouvelle migration pendant le développement, on utilise `npx prisma migrate dev --name <description>`. Prisma compare le `schema.prisma` actuel avec l'état de la base et génère le SQL approprié.

Script de seed

Le fichier `api/src/scripts/seed.js` peuple la base avec des données réalistes au premier démarrage. Le mécanisme repose sur un **utilisateur sentinelle** dont l'email est `seed@etnair.com`. Si cet utilisateur existe déjà au

démarrage, le script considère que la base a déjà été seedée et se contente d'afficher "Base déjà peuplée, seed ignoré" avant de rendre la main. Sinon, il procède au peuplement.

Le seed génère d'abord 35 utilisateurs (25 propriétaires et 10 locataires) avec des prénoms, noms et emails fictifs grâce à la bibliothèque `@faker-js/faker`. Tous ces comptes ont le même mot de passe `password123`, ce qui facilite les tests manuels.

Ensuite, le script crée environ 127 annonces réparties dans 35 villes françaises (Paris, Lyon, Marseille, Bordeaux, Toulouse, Nice, Nantes, Strasbourg, Montpellier, Lille, et bien d'autres). Chaque ville a ses coordonnées GPS de référence, et chaque annonce reçoit ces coordonnées avec un léger jitter aléatoire pour éviter que toutes les annonces d'une même ville se superposent sur la carte. Les titres sont construits à partir d'une liste d'adjectifs (Charmant, Lumineux, Spacieux, etc.) et de types de logements (Studio, T1, T2, Loft, Duplex...) pour donner des résultats du style "Charmant Studio à Lyon" ou "Lumineux T2 à Bordeaux". Les descriptions piochent dans une liste de phrases pré-écrites et y ajoutent une phrase aléatoire Faker.

Si l'on souhaite forcer un re-seed (par exemple parce qu'on a modifié le script), il suffit de supprimer l'utilisateur sentinelle puis de redémarrer le conteneur API :

```
docker compose exec db psql -U etnair_user -d etnair_db -c \  
  "DELETE FROM users WHERE email = 'seed@etnair.com';"  
docker compose restart api
```

Connexion locale

En développement Docker, la base est accessible depuis l'hôte sur le port `5433` (mappé depuis le port `5432` du conteneur). Les identifiants par défaut sont `etnair_user` / `etnair_pass` avec la base `etnair_db`. Depuis un autre conteneur du même réseau Docker (par exemple l'API), il faut utiliser le nom du service `db` à la place de `localhost`.

Une instance **PgAdmin** est également démarrée par Docker Compose et accessible sur `http://localhost:5050`. Le login est `harvey@etnair.xyz` avec le mot de passe `harvey`. Pour s'y connecter à la base, il faut ajouter un nouveau serveur avec les paramètres `host db`, `port 5432`, `user etnair_user`, `password etnair_pass`. PgAdmin permet ensuite d'explorer le schéma, d'exécuter des requêtes SQL ad-hoc et de visualiser le contenu des tables.

Pour les accès en ligne de commande, on peut directement entrer dans le conteneur via `docker compose exec db psql -U etnair_user -d etnair_db`.

Backups

En production sur Kubernetes, un `CronJob` (`k8s/backup-cronjob.yml`) effectue automatiquement une sauvegarde de la base chaque nuit à 2h du matin. Le job lance un conteneur PostgreSQL temporaire qui exécute `pg_dump` et stocke le résultat compressé sur un PVC dédié. Une rotation manuelle est prévue pour supprimer les backups de plus de 30 jours, mais cette politique pourrait être automatisée plus tard.

Bonnes pratiques adoptées

Plusieurs principes guident le travail sur la base de données dans ce projet. **Les requêtes paginées** comme `GET /annonces` utilisent systématiquement `skip` et `take` avec une limite par défaut raisonnable, pour éviter de

surcharger le serveur si quelqu'un demande des milliers de résultats. **Les jointures sont préférées via Prisma** `include` plutôt qu'en faisant des requêtes séparées, ce qui permet de bénéficier des optimisations SQL.

Le soft-delete n'est pas implémenté : les suppressions sont définitives, mais les cascades sont configurées partout où c'est pertinent. Cela simplifie le modèle au prix de la perte d'historique. **Les decimals sont utilisés pour les prix** (`Decimal(10,2)`) plutôt que des floats, pour éviter les erreurs de précision typiques des nombres à virgule flottante. **Les emails et clés métier ont des contraintes d'unicité** au niveau de la base (`@unique`), ce qui garantit la cohérence même si la validation applicative est contournée.